

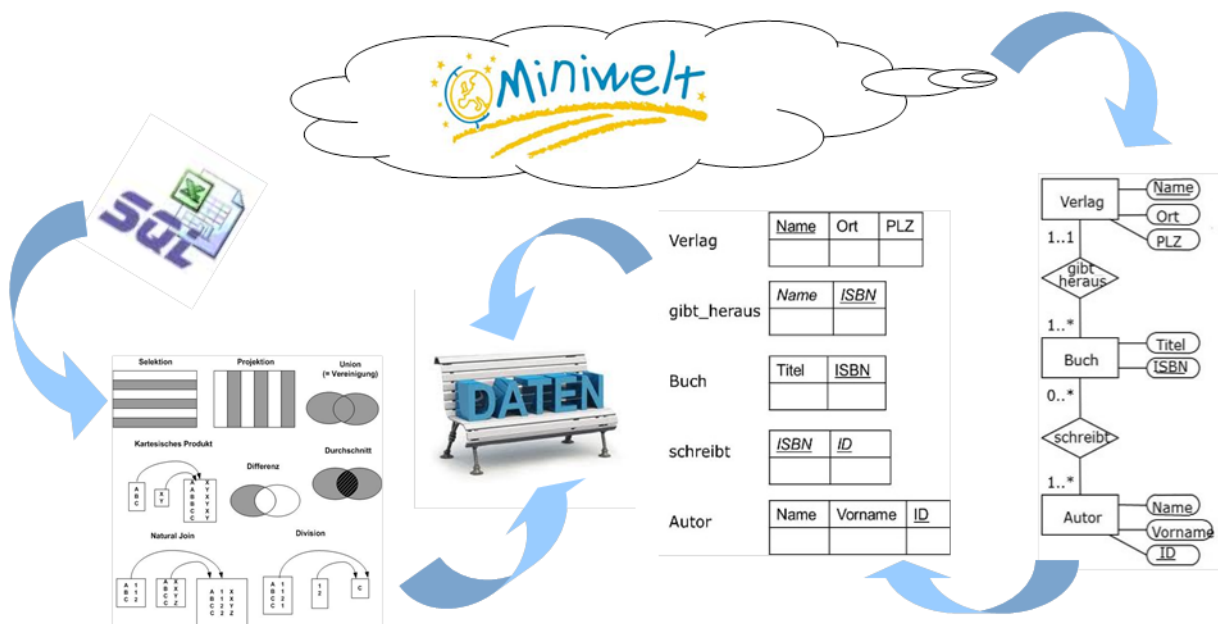
Abgabetermin: 28.5.2026, 12 Uhr
Abgabeform: nur elektronisch

☒ DEM2 G1 Dr. Pitzer Name Klemens Danner Aufwand in h 6
☐ DEM2 G2 Dr. Pitzer
☐ DEM2 G3 Dr. Niklas Punkte _____ Kurzzeichen Tutor _____

Hinweise und Richtlinien:

- Übungsausarbeitungen müssen den im Syllabus angegebenen Formatierungsrichtlinien entsprechen – Nichtbeachtung dieser Formatierungsrichtlinien führt zu Punkteabzug.
- Zusätzlich zu den allgemeinen Formatierungsrichtlinien sind für diese Übungsausarbeitung folgende zusätzlichen Richtlinien zu beachten:
 - Treffen Sie, falls notwendig, sinnvolle Annahmen und dokumentieren Sie diese nachvollziehbar in ihrer Lösung!

Ziel dieser Übung ist es, ausgewählte Bereiche der Anfragesprache SQL praktisch zu vertiefen.



1. Aggregate (Human Resources)

(9 Punkte)

1. Es sollen das höchste Gehalt, das niedrigste Gehalt, die Summe der Gehälter und das Durchschnittsgehalt für alle „Sales Representatives“ (SA_REP) ermittelt werden. Bezeichnen Sie die Spalten entsprechend mit Maximum, Minimum, Summe und Durchschnitt. (1 Punkt)
2. Ändern Sie die Abfrage 1.1, um das niedrigste Gehalt, das höchste Gehalt, die Summe der Gehälter und das Durchschnittsgehalt für jede Job-Kennung (job_id) anzuzeigen. Runden Sie dabei auf ganze Tausender. (1 Punkt)
3. Ermitteln Sie, wie viele unterschiedliche Jobs von Angestellten derzeit besetzt sind. Bezeichnen Sie die Spalte mit Jobanzahl. (1 Punkt)
4. Ermitteln Sie alle Länder-Codes, die mehr als einen Standort haben. Es ist unerheblich, ob an dem Standort Angestellte arbeiten. (1,5 Punkte)
5. Erstellen Sie eine Anfrage, die die Abteilungsnummern, die Abteilungsnamen und die Durchschnittsgehälter für jede Abteilung ermittelt, in der Mitarbeiter*innen angestellt sind. Geben Sie die Durchschnittsgehälter in aufsteigender Reihenfolge aus. (1,5 Punkte)
6. Erstellen Sie eine Anfrage, die die Job-ID und das Maximum der Gehälter für jede Job-ID ermittelt, in denen das Maximum mehr als \$ 10.000 beträgt. (1 Punkt)
7. Erstellen Sie den folgenden Statistikbericht für die Personalabteilung: Geben Sie für alle Angestellte die Manager*in von mehr als einer Person sind die Manager-ID, die Anzahl und das Durchschnittsgehalt dieser Personen aus. Verwenden Sie dazu die GROUP BY-Klausel und die COUNT-Aggregatsfunktion, Runden Sie das Gehalt auf zwei Nachkommastellen, vergeben Sie sinnvolle Namen. (1,5 Punkte)

2. Geschachtelte Anfragen (Human Resources)

(15 Punkte)

1. Erstellen Sie einen Bericht, um die Angestelltennummern und Nachnamen aller Angestellten anzuzeigen, die mehr als das Durchschnittsgehalt verdienen. Sortieren Sie die Ergebnisse in aufsteigender Reihenfolge nach Gehalt. (1 Punkt)
2. Erweitern Sie diesen Bericht, um die Angestelltennummern und Nachnamen aller Angestellten anzuzeigen, die mehr als das Durchschnittsgehalt **in ihrer jeweiligen Abteilung** verdienen. Gehen Sie dabei schrittweise vor: (3 Punkte)
 - a. Erstellen Sie zuerst die Abfrage für das Durchschnittsgehalt einer beliebigen konkreten Abteilung z.B. für Abteilung Nr 10.
 - b. Erstellen Sie dann eine zweite Abfrage für Angestellte deren Gehalt über einem bestimmten Wert liegt, z.B. über \$ 5000.
 - c. Erstellen Sie nun die Anfrage, die die ursprüngliche Aufgabenstellung für alle Mitarbeiter löst unter Zuhilfenahme der Erkenntnisse aus (a) und (b).
3. Geben Sie diejenigen Angestellten aus (Angestelltennummer und Nachname), in deren Abteilung mehr als zwei Personen arbeiten. (1 Punkt)
4. Die Personalabteilung benötigt die Namen der Abteilungen, in denen genau zwei Angestellte arbeiten. Verwenden Sie dazu den IN-Operator. (1,5 Punkte)
5. Erstellen Sie eine Abfrage, um die Angestellte anzuzeigen (last_name, job_id, salary), deren Gehalt höher als das Gehalt aller Programmierer*innen (= 'IT_PROG') ist. Sortieren Sie die Ergebnisse vom höchsten bis zum niedrigsten Gehalt. Verwenden Sie dazu den ALL-Operator. (1 Punkt)
6. Erstellen Sie eine Abfrage, die alle Angestellten (last_name) auflistet, die keine Vorgesetzten sind. Führen Sie diese Aufgabe mit dem Operator NOT EXISTS durch. (1 Punkt)

7. Erstellen Sie eine Abfrage für die Geschäftsführung, die jene Job-Kennung (job_id) mit dem höchsten Durchschnittsgehalt ermittelt. Geben Sie für diesen Job die Job-Kennung und Job-Bezeichnung (job_id, job_title) sowie das Durchschnittsgehalt aus. (1,5 Punkte)
8. Erstellen Sie eine Abfrage, ID, Nachname und Job-Kennung jener Angestellten ermittelt, die mindestens *zweimal* den Job gewechselt haben. (1 Punkt)
9. Erstellen Sie eine Abfrage, die Nachname, Gehalt und Abteilungs-Nummer jener Angestellter ermittelt, die das geringste Gehalt in ihrer Abteilung verdienen. (1 Punkt)
10. Erstellen Sie ausgehend von 2.9 und 2.8 eine Abfrage, die den Nachnamen, die Job-Kennung und das Gehalt jener Angestellten ermittelt, die bereits mindestens *einmal* den Job gewechselt haben und die das geringste Gehalt in ihrer Abteilung verdienen. (1 Punkt)
11. Geben Sie die Vor- und Nachnamen, Gehalt und Manager-ID aller Angestellter aus und sortieren Sie die Ausgabe nach der Differenz des eigenen Gehalts vs. dem Gehalt des/der jeweiligen Manager*in absteigend. Es sollen auch Angestellte ohne Vorgesetzte enthalten sein. (2 Punkte)

3. Zusatzaufgabe (knifflig)

(2 + 2 Punkte)

Die Ausarbeitung dieser Beispiele bringt Ihnen zusätzliche Bonuspunkte.

1. Lösen Sie die Aufgabe 2.6 mit dem NOT IN-Operator!
2. Lösen Sie die Aufgabe 2.10 ohne korrelierte Unterabfragen!


```
SELECT country_id
FROM locations
GROUP BY country_id
HAVING COUNT(location_id) > 1;
```

	COUNTRY_ID
1	US

1.5

Note: selbe department_id und unterschiedlicher department_name ist ausgeschlossen, weil department_id der Primary key von departments ist. Daher kann man nach department_id und department_name gruppieren. (was syntaktisch erforderlich ist, um diese beiden Attribute als Projektion auszugeben)

```
SELECT department_id,  
department_name,  
AVG(salary)  
FROM employees JOIN departments USING(department_id)  
GROUP BY department_id, department_name  
ORDER BY AVG(salary) ASC;
```

[illegible]

1.6

```
SELECT job_id, MAX(salary)
FROM employees
GROUP BY job_id
HAVING MAX(salary) > 10000;
```

	JOB_ID	MAX(SALARY)
1	AD_VP	17000
2	AC_MGR	12000
3	SA_MAN	10500
4	AD_PRES	24000
5	MK_MAN	13000
6	SA_REP	11000

1.7

Code

```
SELECT manager_id AS Manager,
COUNT(*) AS Mitarbeiteranzahl,
ROUND(AVG(salary), 2) AS Durchschnittsgehalt
FROM employees
WHERE manager_id IS NOT NULL
GROUP BY manager_id
HAVING COUNT(*) > 1;
```

Ausgabe

	MANAGER	MITARBEITERANZAHL	DURCHSCHNITTSGEHALT
1	124	4	2925
2	103	2	5100
3	101	2	8200
4	149	3	8866.67
5	100	5	12660

Aufgabe 2

2.1

Code

```
SELECT employee_id, last_name
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
)
ORDER BY salary;
```

Ausgabe

	EMPLOYEE_ID	LAST_NAME
1	103	Hunold
2	149	Zlotkey
3	174	Abel
4	205	Higgins
5	201	Hartstein
6	101	Kochhar
7	102	De Haan
8	100	King

2.2a

Code

```
SELECT AVG(salary)
FROM employees
WHERE department_id = 10;
```

Ausgabe

	AVG(SALARY)
1	4400

2.2b

Code

-- b) Angestellte, die mehr als 4400 verdienen

```
SELECT employee_id, last_name
FROM employees
WHERE salary > 4400;
```

Ausgabe

	EMPLOYEE_ID	LAST_NAME
1	100	King
2	101	Kochhar
3	102	De Haan
4	103	Hunold
5	104	Ernst
6	124	Mourgos
7	149	Zlotkey
8	174	Abel
9	176	Taylor
10	178	Grant
11	201	Hartstein
12	202	Fay
13	205	Higgins
14	206	Gietz

2.2c

Code

-- c) Gemeinsam

```
SELECT employee_id, last_name, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees eSal
    WHERE e.department_id = eSal.department_id
)
ORDER BY salary; -- (Aufgabe 1 erweitern)
```

Ausgabe

	EMPLOYEE_ID	LAST_NAME
1	124	Mourgos
2	103	Hunold
3	149	Zlotkey
4	174	Abel
5	205	Higgins
6	201	Hartstein
7	100	King

2.3

Code

```

SELECT employee_id, last_name
FROM employees e
WHERE 2 < (
    -- Unterabfrage: Anzahl der Angestellten in der Abteilung
    SELECT COUNT(employee_id)
    FROM employees depEmp
    WHERE e.department_id = depEmp.department_id
);

```

Ausgabe

	EMPLOYEE_ID	LAST_NAME
1	100	King
2	101	Kochhar
3	102	De Haan
4	103	Hunold
5	104	Ernst
6	107	Lorentz
7	124	Mourgos
8	141	Rajs
9	142	Davies
10	143	Matos
11	144	Vargas
12	149	Zlotkey
13	174	Abel
14	176	Taylor

2.4

Code

```

SELECT department_name
FROM departments
WHERE department_id IN (
    -- Liste aller department_ids mit genau 2 mitarbeitern
    SELECT department_id
    FROM employees
    GROUP BY department_id
    HAVING COUNT(*) = 2
);

```

Ausgabe

	DEPARTMENT_NAME
1	Accounting
2	Marketing

2.5

Code

```

SELECT last_name, job_id, salary
FROM employees
WHERE salary > ALL (
  SELECT salary
  FROM employees
  WHERE job_id = 'IT_PROG'
)
ORDER BY salary DESC;

```

Ausgabe

	LAST_NAME	JOB_ID	SALARY
1	King	AD_PRES	24000
2	De Haan	AD_VP	17000
3	Kochhar	AD_VP	17000
4	Hartstein	MK_MAN	13000
5	Higgins	AC_MGR	12000
6	Abel	SA_REP	11000
7	Zlotkey	SA_MAN	10500

2.6

Code

```

SELECT last_name
FROM employees e
WHERE NOT EXISTS (
  SELECT manager_id
  FROM employees m
  WHERE e.employee_id = m.manager_id
);

```

Ausgabe

	LAST_NAME
1	Ernst
2	Lorentz
3	Rajs
4	Davies
5	Matos
6	Vargas
7	Abel
8	Taylor
9	Grant
10	Whalen
11	Fay
12	Gietz

2.7

Code

```

SELECT job_id, job_title, AVG(salary)
FROM employees JOIN jobs USING(job_id)
GROUP BY job_id, job_title
HAVING AVG(salary) >= ALL (
    -- Liste aller Durchschnittsgehälter
    SELECT AVG(salary)
    FROM employees
    GROUP BY job_id
);

```

Ausgabe

	JOB_ID	JOB_TITLE	AVG(SALARY)
1	AD_PRES	President	24000

2.8

Code

```

SELECT employee_id, last_name, job_id
FROM employees e
-- where job_history hat genau 2 einträge zu dem employee
WHERE 2 <= (
    SELECT COUNT(*)
    FROM job_history jh
    WHERE e.employee_id = jh.employee_id
);

```

Ausgabe

	EMPLOYEE_ID	LAST_NAME	JOB_ID
1	101	Kochhar	AD_VP
2	176	Taylor	SA_REP
3	200	Whalen	AD_ASST

2.9

Code

```

SELECT last_name, salary, department_id
FROM employees e
WHERE salary = (
    SELECT MIN(salary)
    FROM employees empDep
    WHERE empDep.department_id = e.department_id
);

```

Ausgabe

	LAST_NAME	SALARY	DEPARTMENT_ID
1	Kochhar	17000	90
2	De Haan	17000	90
3	Lorentz	4200	60
4	Vargas	2500	50
5	Taylor	8600	80
6	Whalen	4400	10
7	Fay	6000	20
8	Gietz	8300	110

2.10

Code

```

SELECT last_name, job_id, salary
FROM employees e
WHERE salary = (
    SELECT MIN(salary)
    FROM employees empDep
    WHERE empDep.department_id = e.department_id
) AND 1 <= (
    SELECT COUNT(*)
    FROM job_history jh
    WHERE e.employee_id = jh.employee_id
);

```

Ausgabe

	LAST_NAME	JOB_ID	SALARY
1	De Haan	AD_VP	17000
2	Taylor	SA_REP	8600
3	Kochhar	AD_VP	17000
4	Whalen	AD_ASST	4400

2.11

Code

```

SELECT first_name, last_name, e.salary, manager_id
FROM employees e
ORDER BY (
  SELECT m.salary - e.salary
  FROM employees m
  WHERE m.employee_id = e.manager_id
) DESC;

```

Ausgabe

	FIRST_NAME	LAST_NAME	SALARY	MANAGER_ID
1	Steven	King	24000	(null)
2	Kevin	Mourgos	5800	100
3	Eleni	Zlotkey	10500	100
4	Jennifer	Whalen	4400	101
5	Michael	Hartstein	13000	100
6	Alexander	Hunold	9000	102
7	Lex	De Haan	17000	100
8	Neena	Kochhar	17000	100
9	Pat	Fay	6000	201
10	Shelley	Higgins	12000	101
11	Diana	Lorentz	4200	103
12	William	Gietz	8300	205
13	Kimberely	Grant	7000	149
14	Peter	Vargas	2500	124
15	Randall	Matos	2600	124
16	Bruce	Ernst	6000	103
17	Curtis	Davies	3100	124
18	Trenna	Rajs	3500	124
19	Jonathon	Taylor	8600	149
20	Ellen	Abel	11000	149

Aufgabe 3

3.1

Code

```
SELECT last_name
FROM employees
WHERE employee_id NOT IN (
    SELECT DISTINCT manager_id
    FROM employees
    WHERE manager_id IS NOT NULL
);
```

Ausgabe

	LAST_NAME
1	Ernst
2	Lorentz
3	Rajs
4	Davies
5	Matos
6	Vargas
7	Abel
8	Taylor
9	Grant
10	Whalen
11	Fay
12	Gietz

3.2

Code

```
SELECT e.last_name,
e.job_id,
e.salary
FROM employees e JOIN
(
    SELECT department_id, MIN(salary) AS min_sal
    FROM employees
    GROUP BY department_id
) dep_min ON e.department_id = dep_min.department_id AND
e.salary = dep_min.min_sal
JOIN (
    SELECT employee_id
    FROM job_history
    GROUP BY employee_id
) jChange ON jChange.employee_id = e.employee_id;
```

Ausgabe

	LAST_NAME	JOB_ID	SALARY
1	Kochhar	AD_VP	17000
2	De Haan	AD_VP	17000
3	Whalen	AD_ASST	4400
4	Taylor	SA_REP	8600